# Minimum Swaps to Sort by Digit Sum

## Problem

```
You are given an array nums of distinct positive integers. You need to sort the array in increasing o

Return the minimum number of swaps required to rearrange nums into this sorted order.

A swap is defined as exchanging the values at two distinct positions in the array.

Example 1:
Input: nums = [37,100]
Output: 1
Explanation:
Digit sums: [10, 1]
Sorted order: [100, 37]
Swap 37 with 100. Answer: 1.

Example 2:
Input: nums = [22,14,33,7]
Output: 0
Explanation:
Digit sums: [4, 5, 6, 7]
The array is already sorted. Answer: 0.

Example 3:
Input: nums = [18,43,34,16]
Output: 2
Explanation:
Digit sums: [9, 7, 7, 7]
Sorted order: [16, 34, 43, 18]
Swap 18 with 16, then swap 43 with 34. Answer: 2.

Constraints:
1 <= nums.length <= 10^5
1 <= nums[i] <= 10^9
nums consists of distinct positive integers.
```

## Solution

```
Approach:
1. For every number, compute its digit sum.
2. Sort the numbers by (digit sum, number value). Since all numbers are distinct, this gives a unique
3. Build a mapping from each original number to its target index.
4. Traverse the permutation and count its cycles.
5. A cycle of length L requires L - 1 swaps. Therefore, the minimum number of swaps is n - number_of_

Why cycles work:
A permutation can be decomposed into disjoint cycles. Each cycle can be fixed using exactly one fewer

Example:
nums = [18, 43, 34, 16]
Digit sums: [9, 7, 7, 7]
Target order: [16, 34, 43, 18]

Permutation cycles:
18 -> target index 3 -> 16 -> target index 0 -> 18
43 -> target index 1 -> 34 -> target index 2 -> 43

There are 2 cycles, so swaps = 4 - 2 = 2.
```

## Java Implementation

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

class Solution {
    public int minSwaps(int[] nums) {
        int n = nums.length;
        int[][] sorted = new int[n][2];

        for (int i = 0; i < n; i++) {
            sorted[i][0] = digitSum(nums[i]);
            sorted[i][1] = nums[i];
        }

        Arrays.sort(sorted, (a, b) -> {
            if (a[0] != b[0]) {
                return Integer.compare(a[0], b[0]);
            }
            return Integer.compare(a[1], b[1]);
        });

        Map<Integer, Integer> targetIndex = new HashMap<>();
        for (int i = 0; i < n; i++) {
            targetIndex.put(sorted[i][1], i);
        }

        boolean[] visited = new boolean[n];
        int swaps = n;

        for (int i = 0; i < n; i++) {
            if (visited[i]) {
                continue;
            }

            swaps--;

            int current = i;
            while (!visited[current]) {
                visited[current] = true;
                current = targetIndex.get(nums[current]);
            }
        }

        return swaps;
    }

    private int digitSum(int number) {
        int sum = 0;
        while (number > 0) {
            sum += number % 10;
            number /= 10;
        }
        return sum;
    }
}
```

## Complexity

Time Complexity: O(n log n), dominated by sorting. Space Complexity: O(n), for the sorted pairs, target-index map, and visited array.